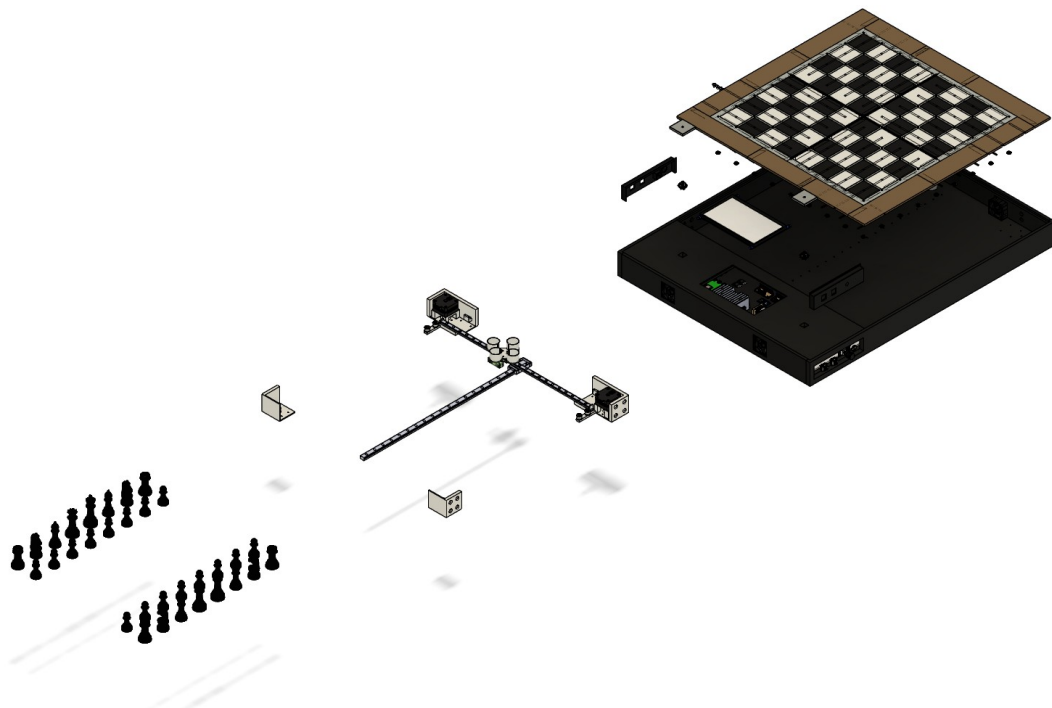


Automated Chessboard: A Distributed IoT System with H-Bot Kinematics



Abstract

This report describes the design and implementation of a smart robotic chessboard utilizing an H-Bot gantry system. The system utilizes an H-Bot gantry architecture constrained to an internal 45mm height profile, orchestrating movement via a distributed network of microcontrollers and a single-board computer. Key contributions include a low-friction material interface for 3D-printed surfaces, a custom JSON-over-UART communication protocol, and a local, offline software stack featuring real-time WebSockets and voice recognition.

Introduction

Traditionally, chess is a game played between two human players. Recent technological advances have enabled to play against chess engines or other humans online. This enables to play chess whenever one likes. However, many people feel a lack of haptic feedback when moving pieces and screens can be exhausting when used over long periods of times. This was the reason for the developement of smart chess boards. The first step were boards that were able to calculate follow up moves, but changes in the pieces position had to be manually registered by pressing buttons. A novelty were LED chess boards that feautered sensor technology and feedback through LED lights inside of the squares. The most modern form of a smart chess board are fully automatic boards which in addition to sensors also have a mechanical system that moves the chess pieces. This is often achieved by a moving electromagnet and magnets in the bottom of the chess pieces that also trigger reed or hall effect sensors.

Many adaptations from makers all around the world have been made for afore mentioned technology. However most self built gantry systems feature an exterior height of more than 100mm making it impractical to use these boards on common tables. An additional problem is that the motors driving the mechanism can be quite loud. Sadly many makers don't document their build process well and only release parts of their work, if at all.

Naturally there also are commercial automatic chessboards. Commercial boards can offer premium technology like chess pieces that have tiny motors built inside of them, individual piece recognition through NFC,RFID or LC circuits and access to otherwise private APIs of popular plattformen. Accompaning those advantages often are very closed ecosystems and great accuisition costs. In some case even subscription models.

The automatic chess board described in this report aims to be an open source plattform that can offer all the features a chess player would want to have in their board. It also aims to be buildable at a modest price that still enables good quality. This is achieved by taking inspiration from existing commercial and self built smart chess boards and innovating on top of them. As soon as the first final version of the chess board will be finished all CAD files of the 3d printed parts, the BOM, a devlog and the whole codebase will be published on GitHub.

Related Work

Many adaptations of the automatic chess board exist. Listed here are the most influential on this project.

Group A: *Haptic/LED Only (Manual Movement)*

ZyChess (Zyger)

The video about this project by Zyger was what inspired the making of this project in the first place. Zyger built a LED smart chessboard as an university project. It didn't feature any sensors, inputs had to be made by buttons. Zyger appeared to be heavily inspired by an [instructable](#) posted by Lewis "Diy Machines".

Custom Electric Chessboard (Olivier Mercier)

Olivier Mercier customized a store bought chessboard to feature sensors and LED lights. Olivier used the Lichess API to play against engines or players, which was controlled by an external touch display. It inspired the placement of the LEDs around the squares instead of in the middle of them.

Group B: *Automated/Robotic Systems*

Phantom Chessboard

Coming in at a retail price of 800 Dollars this board was first developed by a maker before reaching 2 million dollars in funding on kickstarter. Phantom aims to provide a premium experience. It inspired this project to start with a SCARA mechanism before switching to a H-bot mechanism due to scarcity of parts and height of the mechanism.

Gambit

Gambit aims to be an affordable self built chess board and was inspired by Phantom. Gambit changed the SCARA mechanism to a H-bot mechanism, an adaptation of which is used in this project. Gambit posted their progress on Hackaday, still a lot of the files and progress reports are missing, which ended up being a great frustration to this project.

Automated-Chessboard (Greg06)

This self built project incorporates a CoreXY Gantry System to automatically move the chess pieces. It features a button triggered turn system, a small screen and Staunton 3 sized chess pieces. The case of this project was created out of foamboard. Documentation is well written and complete, it even explains some of the theory behind mechanisms and software. "Automated-Chessboard" uses reed switches instead of hall effect sensors and a single Arduino Nano instead of two separate microcontrollers with one single board computer as an orchestrator.

The main improvement on this design in this project is the lower vertical form factor (60%). The screen and additional computing power are also advantages but increase the footprint of the project.

Problem Statement

The primary objective of this project is to develop an open-source, reproducible automated chessboard that balances high-fidelity user experience with cost-effective manufacturing.

Mechanical Strategy & Motion Kinematics

- **Height Limitation:** A critical design constraint is the vertical form factor, which must remain below 60mm—a significant reduction compared to standard CoreXY systems which typically range from 100mm to 120mm.
- **Homing:** The position of the piece moving carriage has to be known
- **Pathfinding Complexity:** Implementing the Knight's jump and Castling requires complex pathfinding algorithms to navigate around occupied squares, as physical pieces cannot "jump" over one another in a 2D plane. Castling introduces a multi-step execution requirement where two pieces must be moved sequentially without collision.
- **Piece Management:** When capturing pieces the captured piece has to move off the board first and be put alongside other captured pieces without them being in the way of the game. Automated capture logic requires a dedicated "graveyard" zone and a non-obstructive path to exit the 64-square play area.
- **Heat Management:** A closed case will retain heat well.
- **Friction:** Moving PETG chess pieces across a textured surface of the same material has to overcome a lot of friction.

Electronical Architecture & Signal Integrity

- **Signal Identification:** Utilizing a Hall Effect sensor matrix allows for binary occupancy detection but introduces the challenge of state tracking, as the sensors cannot natively distinguish between piece types.
 - Hall Effect Sensors often have Open Drain outputs. A microcontroller will only catch electrical noise as the wire will act like an antenna.
- **I/O Multiplexing:** With 64 sensor inputs and limited GPIO pins on the ESP32/Pi, an efficient multiplexing or shift-register architecture is required.
- **Power Integrity:** The system must manage a "noisy" electrical environment where inductive loads (NEMA motors and electromagnets) create back-EMF spikes that threaten sensitive 3.3V logic and LED signal integrity.
- **Voltage Domain Management:** The architecture must efficiently step down 12V (motors/magnets) to 5V (LEDs/Sensors) and 3.3V (Logic) while maintaining a common ground.
- **Electromagnets:** The electromagnets operate at 12V but a GPIO pin can provide 3.3V
- **Motor Driver Communication:** modern motor drivers only have a single UART pin but most microcontrollers have separate rx and tx. If the two pins were just connected to one pin the microcontroller could short circuit its own rx pin when the driver would talk back whilst sending.

Logic Flow & Data Management

- **Compute Orchestration:** The system must handle high-compute tasks (chess engine, Voice Recognition) alongside low-latency hardware interrupts for motor control.
 - **Node Communication:** The main orchestrator must be able to communicate effectively to and from its nodes
- **Latency & Connectivity:** To ensure a "reactive" feel, the system must bypass traditional high-latency HTTP protocols that break the immersion of the "live" board.
- **Sovereign Operation:** The board must support offline edge-computing for privacy and usability in environments without internet access.
- **Data Persistence:** In a multi-user environment, the system must manage conflicting preferences and maintain long-term game records.
- **Online Play:** Some users might wish to remotely play other humans, public puzzles or specific bots they already are familiar with.

Technical Solution

The detailed documentation of the development process aims to achieve a good open source documentation. To provide a good user experience a lot of customizability is given through settings. By moving most settings to a web user interface, cluttering of the main 7" screen is avoided. The screen UI can be controlled by both touch and buttons. Also a chess clock is directly integrated into the side bay of the chess board.

To keep manufacturing and prototyping costs low many components are 3d printed.

Mechanical Strategy & Motion Kinematics

- **H-Bot Gantry Architecture:** To meet the strict <60mm height constraint, an H-Bot kinematics system is utilized. Unlike traditional XY gantries, the H-Bot keeps motors stationary at the frame corners, allowing for a significantly thinner carriage profile and simplified belt routing in a single horizontal plane. It is homed with a limit switch that is mounted on the motor bracket.
- **Collision-Aware Pathfinding:** To handle the "Knight" and "Castling" maneuvers, the system employs a coordinate-based pathfinding algorithm. This logic calculates "indirect" paths—moving pieces along the borders of squares—to avoid collisions with stationary pieces, effectively simulating a "jump" in a 2D physical space.
- **Chess Piece Size:** To enable the afore mentioned pathfinding algorithm piece size was chosen at 80% tournament size. This way most pieces are smaller than half the size of the squares.
- **Automated Displacement Logic:** For piece captures, a dedicated sub-routine calculates a trajectory to the "graveyard" zone. The algorithm prioritizes paths through empty squares to ensure the captured piece does not disturb the active game state.
- **Heat Management:** To provide airflow 40mm server fans can be used
- **Friction:** The use of a PTFE Base underneath each chess piece will provide a durable low friction surface.

Electronical Architecture & Signal Integrity

- **Distributed Processing Model:** The system architecture is divided into three functional domains: an Orchestrator for high-level logic, a Motion Controller for deterministic timing, and a Sensing Node for occupancy detection. This separation prevents "blocking" during heavy AI computations.
 - **Node Communication: UART PROTOCOL TOD0000000000000000**
- **Software-Defined Piece Identification:** Since Hall Effect sensors only provide binary occupancy data, the system maintains a Digital Twin of the board state. By comparing the "before" and "after" sensor matrix scans, the logic identifies which specific piece was moved based on the starting coordinate.

- **Multiplexed Sensing Matrix:** To overcome I/O limitations, the 64 sensors are organized into a scanned grid. This allows for a full board update using significantly fewer microcontroller pins than a direct-mapped approach.
- **Pull-up Resistors:** To deal with the open drain output a pull-up resistor is connected to the line. This way a power signal will be pushed to 5V upon high impedance. The signal is then directed through a logic-level shifter to the microcontroller.
- **Inductive Load Isolation:** To protect sensitive logic, the power architecture implements a tiered regulation system. Inductive components (motors and magnets) are isolated from the logic rail using protection diodes and filtering capacitors to suppress back-EMF spikes.
- **Voltage Domain Management:** To step down from 12V to 5V a buck converter is used. Both the single board computer and microcontrollers step down from 5V to 3.3V. To enable the microcontrollers to read the 5V sensor output, logic level shifters are used. This also enables the fans to receive a 5V PWM signal from a 3.3V GPIO pin.
- **Electromagnets:** To provide 12V the GPIO Pin is wired to a MOSFET which then switches power to the electromagnet. To have the default state of the MOSFET on off, a pull-down resistor is also connected to it. This prevents the electromagnet to accidentally fire during booting when the GPIO pins are in a high-impedance state.
- **Motor Driver Communication:** By placing a 1k resistor on the rx pin the circuit is protected. This enables half-duplex communication on a single pin on the motor driver.

Logic Flow & Data Management

- **Asynchronous Event Loop:** The central software architecture is built on an event-driven loop. This allows the system to poll sensors, process voice inputs, and update the user interface concurrently without serial bottlenecks.
- **Persistent Bi-directional Communication:** To eliminate the latency inherent in request-response models, the system establishes a persistent "tunnel" for data. This ensures that a physical move is reflected on the digital interface in near real-time.
- **Edge-Computing Architecture:** All critical functions—including the chess engine and voice-to-command interpretation—are processed locally on the hardware. This ensures operational sovereignty and eliminates dependencies on external cloud services or internet connectivity.
- **Relational Data Modeling:** A structured data model is used to separate user preferences from game state. This allows for multi-user support where individual settings (like motor speed or difficulty) are stored independently of the ongoing game history.
- **Online Play:** One of the biggest online chess providers offers a very capable API.

Implementation

The publication platform for the open source code will be git hub. It provides neatless integration with git, which will be used with this project, and has a large user base.

The button layout is two buttons on the top of the sidebay and two additional ones on each side as well as one rotary encoder with a click function on each side. This was chosen to provide a direct path to the buttons on the top, as they will control the chess clock, whilst preserving a lot of button functionalities for further options.

To preserve structural integrity with 3d printed parts PETG filament was chosen as it is considered stronger than PLA.

Mechanical Strategy & Motion Kinematics

- **H-Bot**
 - **Motor Choice:** Nema17 0.7A stepper motors were chosen for their size, strength and precision. To control the stepper motors TMC2226 drivers are used. They are silent and have better thermal values than TMC2209.
 - **Pulleys&Idlers Choice:** Matching the common gt2 6mm belt, 20T gt2 6mm pulleys are fitted to the stepper motors. The idlers which are mounted on the 3d printed brackets direct the teathed side of the belt by 180 degrees. Therefore teathed 20T W6 B5 idlers were chosen which are bigger than the smooth 16T W6 B3 idlers on the end of the upper linear guide rail. The smaller idlers meet the smooth side of the belt and direct it by only 90 degrees.
 - **Linear Guide Rail Choice:** For the base a 500mm MGN12H Rail is used. The upper rail is a 450mm MGN9H rail. This choice was made in reference to the Gambit project which successfully integrated a similar mechanism with these rails. The H-Bot of this project is slightly larger because this board uses a 5mm space for the LEDs between each square. Increasing the rail size by 50mm.
- **Pathfinding:** A specific pathfinding mechanism has not yet been chosen because of the project currently being in the beginning of software development.
- **"Graveyard":** To provide space for captured pieces the chess board features a 3d printed brim around the playing area. Along the long sides its size is 51.5mm, along the short sides 48.5mm.

Electronical Architecture & Signal Integrity

- **Distributed Processing Model:** For the calculation heavy single board computer a Raspberry Pi 4B 8GB is used for its power and price effectiveness. Two esp32s were chosen as hardware controllers. Mainly due to pin count and popularity. One of them is responsible for the multiplexed hall effect sensors, the buttons and rotary encoders as well as the LEDs. The other esp32 controls the stepper motors and electromagnets as well as the fans.
- **Hardware**
 - **Electromagnets:** To provide a lot of magnetic force, which drastically decreases with space, P25/20 electromagnets were chosen. They provide up to 8kg of magnetic force.
 - **Hall Effect Sensors:** A digital hall effect sensor, like the A3144 provides better options to the project and is easier to work with than an analog one.
 - **Multiplexers:** using an analog multiplexer is easy with microcontrollers and compatible with all technologies used in this project.
 - **Logic Level Shifters:** The TXS0108E is a bi-directional logic level converter with enough channels and also allows the control of the fans by stepping up from 3.3V to 5V.
 - **LEDs:** WS2812Bs are at a manageable logic level and are common in self built projects
 - **Microphone:** The MS3625 was selected primarily for its ultra-compact surface-mount footprint.
 - **Wires:** 18WAG is used for the main power line before reaching the buck converter from which on most wire is 24AWG, with exception for the sensor data lines that use 30AWG. These diameters should not limit any components from power and are all silicone coated to be flexible.
 - **Fans:** The 40mm Arctic S4028-6K are small, quiet and provide a lot of airflow. They also are cost effective.
-

Logic Flow & Data Management

- **Asynchronous Event Loop**
 1. **BOOT** (system startup, hardware checks, homing motors)
 2. **IDLE/MENU** (Waiting for user login or game start)
 3. **HUMAN_TURN** (Waiting for physical sensor changes)
 4. **ROBOT_TURN** (Calculating and executing mechanical movements)
 5. **ERROR** (illegal move detected or hardware fault)
 6. **GAME_OVER** (display results, save stats)
- **Logic Flow:** The Sensing Node polls the Hall Effect matrix every 100ms. If a delta is detected, a JSON packet is sent to the Orchestrator, which verifies the move against the Stockfish-validated internal board state.

- **Persistent Bi-directional Communication:** A specific WebSocket Solution has not yet been chosen.
- **Edge-Computing Architecture:** To calculate the next move for the robot player Stockfish has been chosen as an engine. It is the most capable and popular engine and has adaptations for the Raspberry Pi.
- **Relational Data Modeling:** The user and game history database will run on SQLite. The Database will be relatively small and SQLite is not as compute heavy and complex as MySQL.
- **Online Play:** Lychess can be accessed through the berserk library in Python.

Discussion

The current state of the project is at the start of writing the software, which is why some software solutions were not yet fully describable in the implementation.

Because Gambit didn't specify many of their mechanical components certain assumptions had to be made. This means that the H-Bot assembly may need some prototyping before being able to be implemented fully.

The development of the H-Bot Smart Chessboard has highlighted several critical engineering trade-offs, particularly regarding manufacturing precision and system integration.

- **Manufacturing Challenges:** The 3D printing of the large frame components encountered significant thermal warping. This led to the partial failure of several dovetail connectors. While the initial prototype was salvaged using adhesive bonding along a straight-edge, this highlighted a limitation in the current design's print orientation. For future iterations, a segmented print strategy or the use of an enclosed printing environment for PETG would be required to maintain the tolerances necessary for the dovetail joints.
- **Mechanical Integrity:** The use of high-strength neodymium magnets for the side-bay assembly proved excellent for rapid prototyping and internal access. However, this choice compromises the structural "rigidity" required for transport; the lack of mechanical fasteners means the board cannot be lifted from the base without risking a collapse of the side walls. A hybrid approach using recessed 3x20mm wood screws is proposed for the final production model.
- **Budgetary Analysis:** The project exceeded the initial target budget of 300 CHF, reaching a total expenditure of approximately 500 CHF. This increase was driven by not incalculating some of the expenditures for tools, wires, filament, unforeseen need of hardware like threaded inserts, resistors, capacitors, mosfets and similar. Despite this 66% increase, the system remains significantly more cost-effective than commercial automated boards, which often retail for 1,000 CHF or more.
- **Software Complexity:** The transition from hardware assembly to software implementation revealed a higher level of complexity than initially anticipated, particularly regarding the implementation of the asynchronous event loop in code and working across multiple files and languages.

7.2 Conclusion

This project successfully demonstrated that a customazible automated chessboard can be housed within a compact 60mm vertical profile. A 40-50% reduction compared to standard CoreXY alternatives. The integration of an H-Bot gantry, a distributed processing model, and low-friction PTFE interfaces provides a viable foundation for a silent, responsive robotic system.

Attachements

